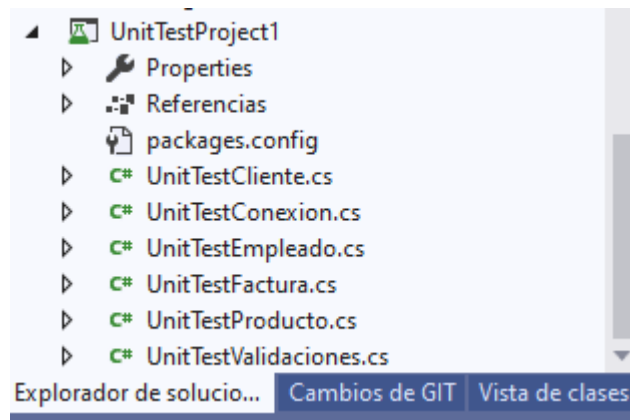


PRUEBAS UNITARIAS

Se realizaron una prueba de unitarias para comprobar el correcto funcionamiento de las diferentes funciones que pueden ser realizadas dentro del sistema. Para ejecutar cada una de las actividades de tal forma que se simule el trabajo real se definieron pruebas para cada componente. En la siguiente figura se puede visualizar la estructura de la solución de “MsTestFramework”.



Cómo se visualiza existe una clase de pruebas unitarias específicas para cada una de las entidades de alta importancia en el sistema. Entre ellas, cliente, empleado, factura, producto. Además, de pruebas de conexión y validación de datos como la cédula y contraseñas.

Con lo ya especificado se procede a especificar el contenido de cada una de estas clases y el objetivo de cada una de las pruebas.

1. PRUEBAS UNITARIAS PARA VALIDACIÓN DE DATOS

La implementación de las pruebas unitarias para validación de datos en el proyecto se centra en evaluar el cumplimiento de las restricciones de la cédula y contraseñas seguras, como se muestra a continuación.

```

public class UnitTestValidaciones
{
    [DataTestMethod]
    [DataRow("Password123!")]
    [DataRow("Contra:2023")]
    [DataRow("ContraseñaD2#")]
     | 0 referencias
    public void MT_Contrasenia_valida(string contrasenia)
    {
        Assert.IsTrue(Validaciones.esContraseniaValida(contrasenia));
    }

    [DataTestMethod]
    [DataRow("Password123")]
    [DataRow("Contra2023")]
    [DataRow("ContraseñaD2")]
    [DataRow("!223123124@")]
     | 0 referencias
    public void MT_Contrasenia_no_valida(string contrasenia)
    {
        Assert.IsFalse(Validaciones.esContraseniaValida(contrasenia));
    }

    [DataTestMethod]
    [DataRow("1234569874")]
    [DataRow("2350703385")]
    [DataRow("1705236521")]
    [DataRow("1705236522")]
    [DataRow("1705236523")]
    [DataRow("1725231524")]
     | 0 referencias
    public void MT_Cedula_valida(string cedula)
    {
        Assert.IsTrue(Validaciones.esCedulaValida(cedula));
    }

    [DataTestMethod]
    [DataRow("123456987")]
    [DataRow("23507033865")]
    [DataRow("17052E6521")]
    [DataRow("1@05236522")]
    [DataRow("170Q23652E")]
    [DataRow("17231524")]
     | 0 referencias
    public void MT_Cedula_no_valida(string cedula)
    {
        Assert.IsFalse(Validaciones.esCedulaValida(cedula));
    }
}

```

2. PRUEBAS UNITARIAS DE CONEXIÓN

La prueba unitaria para la conexión tiene como objetivo comprobar que la conexión pueda abrirse y cerrarse correctamente evitando errores por posibles conexiones abiertas.

[TestClass]

```
public class UnitTestConexion
{
    [TestMethod]
    public void TM_Conexion_abierta()
    {
        Conexion con = new Conexion();
        con.AbrirConexion();

        Assert.IsTrue(con.ConexionAbierta());
        con.CerrarConexion();
    }

    [TestMethod]
    public void TM_Conexion_cerrada_sin_iniciar()
    {
        Conexion con = new Conexion();
        Assert.IsFalse(con.ConexionAbierta());
    }

    [TestMethod]
    public void TM_Conexion_cerrada()
    {
        Conexion con = new Conexion();
        con.AbrirConexion();
        con.CerrarConexion();

        Assert.IsFalse(con.ConexionAbierta());
    }
}
```

3. PRUEBAS UNITARIAS PARA ENTIDAD CLIENTES

Las pruebas unitarias para cada una de las entidades contienen el flujo completo para insertar, modificar, buscar, obtener y eliminar un registro de la base de datos. A continuación, se muestra el contenido.

```
[TestMethod]
0 | 0 referencias
public void TM_1_Insertar_cliente()
{
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = new Cliente();

        cliente.Cedula_cliente = "0000000000";
        cliente.Apellidos = "Apellidos";
        cliente.Nombres = "Nombres";

        Respuesta r = clienteCtrl.insertarCliente(cliente);
        Assert.IsTrue(r.Completado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

```
[TestMethod]
0 | 0 referencias
public void TM_2_Buscar_cliente()
{
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();

        List<Filtros> filtros = new List<Filtros>();
        filtros.Add(new Filtros("campo_busqueda", "cedula_cliente"));
        filtros.Add(new Filtros("texto_buscar", "0000000000"));

        List<Cliente> lstCliente = clienteCtrl.buscarClientes(1, 30, Filtros.filtros_a_xml("Filtros", filtros));
        Assert.IsTrue(lstCliente.Count == 1);
    }
    catch (Exception ex)
    {
        Assert.IsTrue(false);
        throw new Exception($"Error al insertar producto: {ex.Message}");
    }
}
```

[TestMethod]

0 referencias

```
public void TM_3_Get_cliente_por_cedula()
{
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClienteporCedula("0000000000");

        Assert.IsNotNull(cliente);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

[TestMethod]

0 referencias

```
public void TM_4_Get_cliente_por_id()
{
    bool encontrado = false;
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClienteporCedula("0000000000");

        if (cliente != null)
        {
            Cliente c = clienteCtrl.getClienteporId(cliente.Id_cliente);
            encontrado = c != null;
        }

        Assert.IsTrue(encontrado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

[TestMethod]

0 | 0 referencias

public void TM_5_Modificar_cliente()

```
{
    bool modificado = false;
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientPorCedula("0000000000");

        if (cliente != null)
        {
            cliente.Nombres = "NombresMod";
            cliente.Apellidos = "ApellidosMod";

            Respuesta r = clienteCtrl.modificarCliente(cliente);
            modificado = r.Completado;
        }

        Assert.IsTrue(modificado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

[TestMethod]

0 | 0 referencias

public void TM_6_Eliminar_cliente()

```
{
    bool eliminado = false;
    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = clienteCtrl.getClientPorCedula("0000000000");

        if (cliente != null)
        {
            Respuesta r = clienteCtrl.eliminarCliente(cliente.Id_cliente);
            eliminado = r.Completado;
        }

        Assert.IsTrue(eliminado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

4. PRUEBAS UNITARIAS PARA ENTIDAD PRODUCTOS

[TestMethod]

```
public void TM_1_Insertar_producto()
{
    try
    {
        ProductoCtrl productoCtrl = new ProductoCtrl();
        Producto producto = new Producto();

        producto.Nombre_producto = "00Producto00";
        producto.Iva = 0;
        producto.Precio_unitario = 0;

        Respuesta r = productoCtrl.insertarProducto(producto);
        Assert.IsTrue(r.Completado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

[TestMethod]

0 referencias

```
public void TM_2_Buscar_producto()
{
    try
    {
        ProductoCtrl productoCtrl = new ProductoCtrl();

        List<Filtros> filtros = new List<Filtros>();
        filtros.Add(new Filtros("campo_busqueda", "nombre_producto"));
        filtros.Add(new Filtros("texto_buscar", "00Producto00"));

        List<Producto> lstProducto = productoCtrl.buscarProductoPaginacion(1, 30, Filtros.filtros_a_xml("Filtros", filtros));
        Assert.IsTrue(lstProducto.Count == 1);
    }
    catch (Exception ex)
    {
        Assert.IsTrue(false);
    }
}
```

```

[TestMethod]
0 | 0 referencias
public void TM_3_Get_producto()
{
    try
    {
        ProductoCtrl productoCtrl = new ProductoCtrl();

        List<Filtros> filtros = new List<Filtros>();
        filtros.Add(new Filtros("campo_búsqueda", "nombre_producto"));
        filtros.Add(new Filtros("texto_buscar", "00Producto00"));

        Producto producto = productoCtrl.buscarProductoPaginacion(1, 30, Filtros.filtros_a_xml("Filtros", filtros))[0];
        producto = productoCtrl.getProducto(producto.Id_producto);

        Assert.IsNotNull(producto);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

```

[TestMethod]
0 | 0 referencias
public void TM_4_Modificar_producto()
{
    bool modificado = false;
    try
    {
        ProductoCtrl productoCtrl = new ProductoCtrl();
        List<Producto> lstProducto = productoCtrl.buscarProducto("00Producto00");

        if (lstProducto.Count > 0)
        {
            Producto producto = lstProducto[0];
            producto.Nombre_producto = "00Producto00Mod";
            producto.Iva = 0;
            producto.Precio_unitario = 0;

            Respuesta r = productoCtrl.modificarProducto(producto);
            modificado = r.Completado;
        }

        Assert.IsTrue(modificado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```


[TestMethod]

0 referencias

public void TM_5_Eliminar_producto()

```
{
    bool eliminado = false;
    try
    {
        ProductoCtrl productoCtrl = new ProductoCtrl();
        List<Producto> lstProducto = productoCtrl.buscarProducto("00Producto00");

        if (lstProducto.Count > 0)
        {
            Producto producto = lstProducto[0];

            Respuesta r = productoCtrl.eliminarProducto(producto.Id_producto);
            eliminado = r.Completado;
        }

        Assert.IsTrue(eliminado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

5. PRUEBAS UNITARIAS PARA ENTIDAD EMPLEADO

Al igual que las pruebas aplicadas para la entidad empleado, esta contiene las transacciones dml para manipular la información o registros de empleados.

```
[TestMethod]
0 | 0 referencias
public void TM_1_Insertar_empleado()
{
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();

        Empleado empleado = new Empleado();
        empleado.Cedula_empleado = "0000000000";
        empleado.Apellidos = "Apellidos";
        empleado.Nombres = "Nombres";
        empleado.Fecha_nacimiento = new DateTime(1955, 1, 1).ToString();

        Usuario usuario = new Usuario();
        usuario.Tipo_usuario.Id_tipo_usuario = 1;
        usuario.Nombre_usuario = "_Usuario_";
        usuario.Contrasenia = "123";
        usuario.Activo = true;

        Respuesta r = empleadoCtrl.insertarEmpleado(empleado, usuario);
        Assert.IsTrue(r.Completado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

[TestMethod]
0 | 0 referencias
public void TM_2_Buscar_empleado()
{
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();

        List<Filtros> filtros = new List<Filtros>();
        filtros.Add(new Filtros("campo_búsqueda", "cedula_empleado"));
        filtros.Add(new Filtros("texto_buscar", "0000000000"));

        List<Empleado> lstEmpleado = empleadoCtrl.buscarEmpleados(1, 30, filtros.filtros_a_xml("Filtros", filtros));
        Assert.IsTrue(lstEmpleado.Count == 1);
    }
    catch (Exception ex)
    {
        Assert.IsTrue(false);
        throw new Exception($"Error al insertar producto: {ex.Message}");
    }
}
```

[TestMethod]

0 | 0 referencias

public void TM_3_Get_empleado_por_cedula()

```
{
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
        Empleado empleado = empleadoCtrl.getEmpleadoPorCedula("0000000000");

        Assert.IsNotNull(empleado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

[TestMethod]

0 | 0 referencias

public void TM_4_Get_empleado_por_id()

```
{
    bool encontrado = false;
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
        Empleado empleado = empleadoCtrl.getEmpleadoPorCedula("0000000000");

        if (empleado != null)
        {
            Empleado e = empleadoCtrl.getEmpleadoPorId(empleado.Id_empleado);
            encontrado = (e != null);
        }

        Assert.IsTrue(encontrado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

[TestMethod]

0 | 0 referencias

public void TM_5_Modificar_empleado()

```
{
    bool modificado = false;
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
        Empleado empleado = empleadoCtrl.getEmpleadoPorCedula("0000000000");

        if (empleado != null)
        {
            empleado = empleadoCtrl.getEmpleadoUsuarioPorIdEmpleado(empleado.Id_empleado);

            empleado.Nombres = "NombresMod";
            empleado.Apellidos = "ApellidosMod";
            empleado.Usuario.Contrasenia = "123";

            Respuesta r = empleadoCtrl.modificarEmpleado(empleado, empleado.Usuario);
            modificado = r.Completado;
        }

        Assert.IsTrue(modificado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

[TestMethod]

0 | 0 referencias

public void TM_6_Modificar_contrasenia_empleado()

```
{
    try
    {
        UsuarioCtrl usuarioCtrl = new UsuarioCtrl();
        Respuesta r = usuarioCtrl.modificarContrasenia("_Usuario_", "123", "1234");

        Assert.IsTrue(r.Completado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

```

[TestMethod]
0 | 0 referencias
public void TM_7_Modificar_empleado_contraseña_incorrecta()
{
    bool modificado = false;
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
        Empleado empleado = empleadoCtrl.getEmpleadoPorCedula("0000000000");

        if (empleado != null)
        {
            empleado = empleadoCtrl.getEmpleadoUsuarioPorIdEmpleado(empleado.Id_empleado);

            empleado.Nombres = "NombresMod";
            empleado.Apellidos = "ApellidosMod";
            empleado.Usuario.Contrasenia = "123";

            Respuesta r = empleadoCtrl.modificarEmpleado(empleado, empleado.Usuario);
            modificado = r.Completado;
        }

        Assert.IsFalse(modificado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

```

[TestMethod]
0 | 0 referencias
public void TM_8_Eliminar_empleado()
{
    bool eliminado = false;
    try
    {
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
        Empleado empleado = empleadoCtrl.getEmpleadoPorCedula("0000000000");

        if (empleado != null)
        {
            Respuesta r = empleadoCtrl.eliminarEmpleado(empleado.Id_empleado);
            eliminado = r.Completado;
        }

        Assert.IsTrue(eliminado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

6. PRUEBAS UNITARIAS PARA ENTIDAD FACTURA

Para las pruebas unitarias de Factura, se utilizó la notación initialize para registrar un cliente, producto y empleado para poder realizar las acciones dml de la factura. A continuación, se muestra el proceso.

```
[ClassInitialize]
0 referencias
public static void Initialize(TestContext context)
{
    try
    {
        //Insertar un empleado y usuarii para las pruebas
        EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();

        Empleado empleado = new Empleado();
        empleado.Cedula_empleado = "0000000000";
        empleado.Apellidos = "Apellidos";
        empleado.Nombres = "Nombres";
        empleado.Fecha_nacimiento = new DateTime(1955, 1, 1).ToString();

        Usuario usuario = new Usuario();
        usuario.Tipo_usuario.Id_tipo_usuario = 1;
        usuario.Nombre_usuario = "_Usuario_";
        usuario.Contrasenia = "123";
        usuario.Activo = true;

        empleadoCtrl.insertarEmpleado(empleado, usuario);
    }
    catch (Exception ex)
    {
        throw new Exception($"Error al insertar empleado: {ex.Message}");
    }

    try
    {
        ClienteCtrl clienteCtrl = new ClienteCtrl();
        Cliente cliente = new Cliente();

        cliente.Cedula_cliente = "0000000000";
        cliente.Apellidos = "Apellidos";
        cliente.Nombres = "Nombres";

        clienteCtrl.insertarCliente(cliente);
    }
    catch (Exception ex)
    {
        throw new Exception($"Error al insertar cliente: {ex.Message}");
    }

    //Insertar un producto
    try
    {
        ProductoCtrl productoCtrl = new ProductoCtrl();
        Producto producto = new Producto();

        producto.Nombre_producto = "00Producto00";
        producto.Iva = 0;
        producto.Precio_unitario = 0;

        productoCtrl.insertarProducto(producto);
    }
    catch (Exception ex)
    {
        throw new Exception($"Error al insertar producto: {ex.Message}");
    }
}
```

[TestMethod]

0 referencias

public void TM_1_Insertar_factura()

```
{
    try
    {
        FacturaCtrl facturaCtrl = new FacturaCtrl();

        Factura factura = new Factura();
        FacturaDetalle fDetalle = new FacturaDetalle();

        EmpleadoCtrl eCtrl = new EmpleadoCtrl();
        Empleado empleadoPrueba = eCtrl.getEmpleadoPorCedula("0000000000");

        ProductoCtrl pCtrl = new ProductoCtrl();
        Producto productoPrueba = pCtrl.buscarProducto("00Producto00")[0];

        ClienteCtrl cCtrl = new ClienteCtrl();
        Cliente clientePrueba = cCtrl.getClientePorCedula("0000000000");

        factura.Cliente = clientePrueba;
        factura.Empleado = empleadoPrueba;
        fDetalle.Producto = productoPrueba;
        fDetalle.Cantidad = 0;

        factura.Factura_Detalle.Add(fDetalle);

        Respuesta r = facturaCtrl.insertarFactura(factura);

        id_factura = int.Parse(r.Id);
        id_empleado = empleadoPrueba.Id_empleado;
        id_producto = productoPrueba.Id_producto;

        Assert.IsTrue(r.Completado);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}
```

```

[TestMethod]
0 referencias
public void TM_2_Buscar_facturas()
{
    try
    {
        FacturaCtrl facturaCtrl = new FacturaCtrl();

        List<Filtros> filtros = new List<Filtros>();
        filtros.Add(new Filtros("campo_busqueda", "id_factura"));
        filtros.Add(new Filtros("texto_buscar", id_factura.ToString()));
        filtros.Add(new Filtros("fecha_inicio", DateTime.Now.ToString("MM-dd-yyyy")));
        filtros.Add(new Filtros("fecha_fin", DateTime.Now.ToString("MM-dd-yyyy")));

        List<Factura> facturas = facturaCtrl.buscarFacturas(1, 30, filtros.filtros_a_xml("Filtros", filtros));
        Assert.IsTrue(facturas.Count == 1);
    }
    catch (Exception)
    {
        Assert.IsTrue(false);
    }
}

```

```

[ClassCleanup]
0 referencias
public static void Cleanup()
{
    //Eliminar el cliente, producto, empleado y factura utilizado en las pruebas
    //Eliminar factura y su detalle
    FacturaCtrl facturaCtrl = new FacturaCtrl();
    facturaCtrl.eliminarFactura(id_factura);

    //Eliminar el empleado
    EmpleadoCtrl empleadoCtrl = new EmpleadoCtrl();
    empleadoCtrl.eliminarEmpleado(id_empleado);

    //Eliminar el producto
    ProductoCtrl productoCtrl = new ProductoCtrl();
    productoCtrl.eliminarProducto(id_producto);

    //Eliminar el cliente
    ClienteCtrl clienteCtrl = new ClienteCtrl();
    Cliente clientePrueba = clienteCtrl.getClientePorCedula("0000000000");
    clienteCtrl.eliminarCliente(clientePrueba.Id_cliente);
}

```

Finalmente, para eliminar los datos se ejecuta la función Cleanup(), misma que se ejecuta automáticamente al terminar de evaluar cada una de las pruebas unitarias de la clase de Factura.

Así, al ejecutar las pruebas unitarias, se obtienen los siguientes resultados.

Prueba	Duración	Rasgos
▲  UnitTestProject1 (28)	2,3 s	
▲  UnitTestProject1 (28)	2,3 s	
▲  Unit1TestCliente (6)	1,3 s	
 TM_1_Insertar_cliente	685 ms	
 TM_2_Buscar_cliente	140 ms	
 TM_3_Get_cliente_por_cedula	24 ms	
 TM_4_Get_cliente_por_id	282 ms	
 TM_5_Modificar_cliente	60 ms	
 TM_6_Eliminar_cliente	62 ms	
▲  Unit2TestProducto (5)	368 ms	
 TM_1_Insertar_producto	50 ms	
 TM_2_Buscar_producto	26 ms	
 TM_3_Get_producto	43 ms	
 TM_4_Modificar_producto	179 ms	
 TM_5_Eliminar_producto	70 ms	
▲  Unit3TestEmpleado (8)	450 ms	
 TM_1_Insertar_empleado	50 ms	
 TM_2_Buscar_empleado	25 ms	
 TM_3_Get_empleado_por_c...	20 ms	
 TM_4_Get_empleado_por_id	44 ms	
 TM_5_Modificar_empleado	94 ms	
 TM_6_Modificar_contraseñ...	43 ms	
 TM_7_Modificar_empleado_...	64 ms	
 TM_8_Eliminar_empleado	110 ms	
▲  Unit4TestFactura (2)	133 ms	
 TM_1_Insertar_factura	108 ms	
 TM_2_Buscar_facturas	25 ms	
▲  UnitTestConexion (3)	< 1 ms	
 TM_Conexion_abierta	< 1 ms	
 TM_Conexion_cerrada	< 1 ms	
 TM_Conexion_cerrada_sin_i...	< 1 ms	
▲  UnitTestValidaciones (4)	91 ms	
 MT_Cedula_no_valida	71 ms	
 MT_Cedula_valida	< 1 ms	
 MT_Contrasenia_no_valida	20 ms	
 MT_Contrasenia_valida	< 1 ms	